

Code Explanation

Testing DLT Pipeline Code Explanation

This is a test suite written in Python using the Pytest framework to test the functionality of a data pipeline built using Databricks' Delta Live Tables (DLT). The code is designed to test the `customer`, `order`, and `customeraggregatespend` functions defined in the `src/dlt\_pipeline.py` module.

Overview of the Code

The test suite mocks the DLT module and SparkSession to isolate the dependencies and test the functions in the `dlt\_pipeline` module. It uses Pytest fixtures to set up the mocks and test the functions.

Step 1: Importing Modules and Setting Up Mocks

The code starts by importing the necessary modules, including Pytest, tempfile, and PySpark. It also sets up mocks for the DLT module and SparkSession.

```
import pytest
import tempfile
import os
import shutil
from pyspark.sql import SparkSession
from unittest.mock import patch, MagicMock
import sys
```

Step 2: Creating Mock DLT Module

A mock DLT module is created to replace the actual DLT module, which is only available in Databricks. The mock module defines the `table`, `expect\_or\_drop`, and `read` methods.

```
class MockDLT:
    @staticmethod
    def table(*args, **kwargs):
        def decorator(func):
            return func
        return decorator

    @staticmethod
    def expect_or_drop(*args, **kwargs):
        def decorator(func):
            return func
        return decorator

    @staticmethod
    def read(table_name):
        if table_name == "customer":
            return mock_customer_df
        elif table_name == "order":
            return mock_order_df
        elif table_name == "ordersummary":
            return mock_ordersummary_df
        return None
```

Step 3: Creating Mock DataFrames and SparkSession

Mock DataFrames are created for `customer`, `order`, and `ordersummary` tables. A mock SparkSession is also created.

```
mock_customer_df = MagicMock()
mock_order_df = MagicMock()
mock_ordersummary_df = MagicMock()

mock_spark = MagicMock()
```

Step 4: Setting Up Pytest Fixture

A Pytest fixture `setup\_mocks` is defined to set up the mocks for the DataFrames and SparkSession.

```
@pytest.fixture(scope="module")
def setup_mocks():
    mock_customer_df.join.return_value = mock_customer_df
    mock_customer_df.select.return_value = mock_customer_df
    # ... other mock setups ...
    mock_spark.read.option.return_value.option.return_value.csv.return_value = mock_customer_df
    mock_spark.table.return_value = mock_ordersummary_df
```

Step 5: Testing DLT Pipeline Functions

The test suite includes three test functions: `test\_dlt\_customer\_table`, `test\_dlt\_order\_table`, and `test\_dlt\_customeragggregatespend`. Each test function uses the `@patch.dict` decorator to mock the `dlt` module and imports the `dlt\_pipeline` module after mocking.

```
@patch.dict('sys.modules', {'dlt': MockDLT()})
def test_dlt_customer_table(setup_mocks):
    from src.dlt_pipeline import customer
    global spark
    spark = mock_spark
    result = customer()
    mock_spark.read.option.assert_called_with("header", "true")
    assert result is not None
```

Step 6: Verifying Functionality

The test functions verify that the `customer`, `order`, and `customeragggregatespend` functions are working correctly by checking the mock calls and assertions.

```
@patch.dict('sys.modules', {'dlt': MockDLT()})
@patch('src.dlt_pipeline.dlt', new_callable=lambda: MockDLT())
def test_dlt_customeragggregatespend(mock_dlt, setup_mocks):
    from src.dlt_pipeline import customeragggregatespend
    result = customeragggregatespend()
    mock_ordersummary_df.filter.assert_called_once()
    assert result is not None
```